Evolutionary rescue under habitat fragmentation

Jan Leuenberger & Safeeya Ramis

2026-10-28

1. Introduction

This report documents a stochastic simulation of **evolutionary rescue** in discrete time, based on the model of Orr & Unckless introduced in the practical. We extended the basic model with **habitat fragmentation**: at a chosen generation (`split_time`) the population is split into two separate and isolated habitats (B and C) of varieng sizes, each with its own carrying capacity.

Biological scenario

- A haploid population consists of wild-type individuals (**a**) and, possibly, mutant individuals (**A**).
- A sudden environmental change makes the wild type unfit: it **declines** at rate `decay_rate`.
- The mutant is **beneficial** (advantage `sel_coeff`) and can grow in the new environment, but it is initially rare or absent. If absent, it has to arise by mutation from the wild type.
- The population is *rescued* if the mutant spreads before the wild type (and with it the whole population) goes extinct. -The Habitat gets seperatated at a defined time called `split_time`.Individuals are randomly distributed, each habitat has a smaller carrying capacity, and both then evolve independently.

Research Questions

- Is evolutionary rescue possible in the Fragmented habitats?
  - What is the impact of split_time on ER?
  - What is the impact of the habitat porportions on ER?

1.1 Overview of the code

1. `simulate_one_gen()`: simulates **one generation** (reproduction with logistic growth + mutation) for one habitat.
2. `simulate_pop()`: runs a **full simulation**: one habitat until `split_time`, then two habitats (B and C) afterwards.
3. Plot functions (`plot_split()`, `plot_habitat()`, `plot_habitats()`): visualise a single simulation run.
4. `run_scenarios()` and `to_long()`: repeat the simulation many times for different parameter values and summaries the **rescue probability**.
5. Three analyses: split time varies, habitat proportion varies, and both.

1.2 Parameters and variables

Name	Meaning
`N_a`, `N_A`	current size of the wild-type / mutant population (change over time)
`N_init_a`, `N_init_A`	initial sizes of wild type (positive integer) and mutant (non-negative integer)
`decay_rate`	detrimental effect of the environment on the wild type (> 0 so that extinction is possible)
`sel_coeff`	selective advantage of the mutant (should be $>$ `decay_rate` to allow rescue)
`mut_rate`	mutation rate from wild type to mutant (> 0 and small)
`t_max`	maximum number of generations simulated (a setting, not a model parameter)
`split_time`	generation at which the fragmentation happens (integer > 0)
`split_ratio`	probability with which each individual is assigned to habitat B at the split (between 0 and 1)
`habitat_prop`	share of the total carrying capacity given to habitat B (between 0 and 1); habitat C gets `1 - habitat_prop`
`K`	carrying capacity of the original (unsplit) habitat; limits growth and is divided among B and C in the same proportions as the habitats

Simulating evolutionary rescue in discrete time

In this notebook, we are implementing simulations of evolutionary rescue by a single mutation in discrete time, as discussed and analyzed in the work by Orr & Unckless. The model is as follows: a haploid population starts with a given number of wild-type and mutant individuals. Because of a sudden environmental change, the wild type is unfit and declines in the new environment. The mutant is beneficial and can survive and expand in the new environment, but it starts at a low (or zero) frequency in the total population. In order to rescue the population, it has to outcompete the wild type. If the mutant is not present at time 0, it has to appear through mutation and then spread in the population before extinction occurs.

Simulate one generation of population size change

We want to observe the numbers of wild-type and mutant individuals over time. As first step, we simulate one generation of the population. At time t, the population has N_a wild-type individuals and N_A mutant individuals. First, these reproduce. This is modeled by drawing a Poisson random variable according to the expected absolute number $N_a \cdot (1 - r)$ of wild-type individuals, where r is the decay rate of the wild type, and $N_A \cdot (1 - r + s)$ of mutant individuals, where s is the selective advantage of the mutant over the wild time. Then mutation from a to A occurs as a Poisson random variable according to the given mutation rate. This defines the composition of the population at time $t + 1$.

1. **Expected offspring (logistic growth).** Without competition, the per-capita growth rates are

   - wild type: $\rho_a = -r$ (negative, so the type declines), with $r =$ `decay_rate`
   - mutant: $\rho_A = s - r$ (positive if $s > r$), with $s =$ `sel_coeff`

   Competition is added by a **density-dependent term**: the growth factor of each type is reduced by $\rho_A \cdot N_{tot}/K$. The expected number of offspring is therefore

$$\bar{N}_a = N_a \max\!\left(0,\ 1 + \rho_a - \rho_A \tfrac{N_{tot}}{K}\right), \qquad \bar{N}_A = N_A \max\!\left(0,\ 1 + \rho_A - \rho_A \tfrac{N_{tot}}{K}\right)$$

When N_{tot} approaches K, the mutant's growth factor approaches 1 (population stays constant) and the wild type keeps declining. When $N_{tot} > K$ the mutant would shrink back towards K. The `max(0, ...)` prevents negative expected values, which `rpois()` cannot handle.

2. **Demographic stochasticity.** The actual number of offspring is drawn from a **Poisson distribution** whose mean is the expected value above. This is what makes every run different.

3. **Mutation.** Of the wild-type offspring, a Poisson-distributed number (mean `offsp_a * mut_rate`) mutates to type A. These are subtracted from the wild type and added to the mutants.

The function returns a vector `c(N_a_new, N_A_new)`.

```
set.seed(57)

simulate_one_gen <- function(N_a, N_A, decay_rate, sel_coeff, mut_rate, K) {
  # total population size and per-capita growth rates without competition
  N_tot <- N_a + N_A
  rho_a <- -decay_rate
  rho_A <- sel_coeff - decay_rate
  # logistic growth: crowding reduces the growth of both types equally (set to 0 if negative)
  mean_a <- N_a * max(0, 1 + rho_a - rho_A * N_tot / K)
  mean_A <- N_A * max(0, 1 + rho_A - rho_A * N_tot / K)
  # draw offspring according to Poisson distribution
  offsp_a <- rpois(1, mean_a)
  offsp_A <- rpois(1, mean_A)

  # draw new mutants according to Poisson distribution
  mut_a_to_A <- rpois(1, offsp_a * mut_rate)

  # determine new population sizes of wild type and mutant
  N_a_new <- offsp_a - mut_a_to_A
  N_A_new <-  offsp_A + mut_a_to_A

  return(c(N_a_new, N_A_new))
}


# Test the function
print(simulate_one_gen(100,0,0.1,0.2,0.001,1000))
```

```
## [1] 82  0
```

```
simulate_pop <- function(N_init_a, N_init_A, decay_rate, sel_coeff, mut_rate, t_max, K, habitat_prop, sp
  # Create the vector in which to save the results
  pop_vector <- c(N_init_a, N_init_A)
  # initiate the variables
  pop_new <- c(N_init_a, N_init_A)

  # Create the vectors for the new habitats
  pop_vector_B <- NULL
  pop_vector_C <- NULL
```

```
# run the simulation until generation t_max
for (i in 1:t_max) {

   # Time when the fragmentation occur
  if (i == split_time + 1) {

  # Create the new habitats (creating new K)
  K_B <- K * habitat_prop
  K_C <- K * (1 - habitat_prop)

  #create the ratio of the splitting
  split_ratio_B <- split_ratio
  split_ratio_C <- (1 - split_ratio_B)
  # print(split_ratio_B)
  # print(split_ratio_C)
  # Distribute individuals  randomly to the habitat B
  n_B_a <- rbinom(1, pop_new[1], split_ratio_B)
  n_B_A <- rbinom(1, pop_new[2], split_ratio_B)

  # Distribute individuals randomly to the habitat C
  n_C_a <- pop_new[1] - n_B_a
  n_C_A <- pop_new[2] - n_B_A

  # Calculate the new populatins for B an C
  pop_new_B <- simulate_one_gen(n_B_a, n_B_A, decay_rate, sel_coeff, mut_rate, K_B)
  pop_new_C <- simulate_one_gen(n_C_a, n_C_A, decay_rate, sel_coeff, mut_rate, K_C)

  # add the new population sizes to the output vector
  pop_vector_B <- rbind(pop_vector_B,pop_new_B)
  pop_vector_C <- rbind(pop_vector_C,pop_new_C)

  }

  if (i > split_time + 1) {

  pop_new_B <- simulate_one_gen(
    pop_new_B[1], pop_new_B[2],
    decay_rate, sel_coeff, mut_rate,K_B
  )

  pop_new_C <- simulate_one_gen(
    pop_new_C[1], pop_new_C[2],
    decay_rate, sel_coeff, mut_rate,K_C
  )

  # Save the results for each generation
  pop_vector_B <- rbind(pop_vector_B, pop_new_B)
  pop_vector_C <- rbind(pop_vector_C, pop_new_C)

  # define the row and column names of the output vector
  rownames(pop_vector_B) <- (0:t_max)[1:length(pop_vector_B[,1])]
  rownames(pop_vector_C) <- (0:t_max)[1:length(pop_vector_C[,1])]
  colnames(pop_vector_B) <- c("a","A")
```

```
    colnames(pop_vector_C) <- c("a","A")

    }

    # Continue simulating habitat A only until split_time
if (i <= split_time) {

    # redefine the current population one generation later
    pop_new <- simulate_one_gen(pop_new[1],pop_new[2], decay_rate, sel_coeff, mut_rate,K)
    # add the new population sizes to the output vector
    pop_vector <- rbind(pop_vector,pop_new)
    # condition to stop the simulation before t_max: either the population exceeds 2 times the original
    if (pop_new[1]+pop_new[2]>=2*(N_init_a+ N_init_A) | pop_new[1]+pop_new[2]==0) break
}

 }
  # define the row and column names of the output vector
  rownames(pop_vector) <- (0:t_max)[1:length(pop_vector[,1])] # note that the vector has to be cut if t
  colnames(pop_vector) <- c("a","A")

  # Data frames
    pop_vector_df <- as.data.frame(pop_vector)
    pop_vector_B_df <- as.data.frame(pop_vector_B)
    pop_vector_C_df <- as.data.frame(pop_vector_C)
  # return all three population vectors separately
  return(list(main = pop_vector_df, B = pop_vector_B_df, C = pop_vector_C_df))


}
```

```
# ============================================================
# Plot functions for the evolutionary rescue simulation
# with habitat fragmentation
# ============================================================

# ---- 1. Overview plot: wild type and mutant before the split,
#         total of habitats B and C after the split, legend outside ----
plot_split <- function(output, K, habitat_prop, split_time, xlim_max = NULL) {

  main <- output$main
  B    <- output$B
  C    <- output$C

  K_B <- K * habitat_prop
  K_C <- K * (1 - habitat_prop)

  # population sizes before the split (wild type and mutant separately)
  wt_main  <- main$a
  mut_main <- main$A

  # total population sizes after the split
  tot_B <- B$a + B$A
  tot_C <- C$a + C$A
```

```r
  # generation numbers
  gen_main <- 0:(length(wt_main) - 1)
  gen_B    <- split_time + seq_along(tot_B)
  gen_C    <- split_time + seq_along(tot_C)

  # axis ranges
  x_max <- if (is.null(xlim_max)) max(c(gen_main, gen_B)) else xlim_max
  y_max <- max(c(K, wt_main, mut_main, tot_B, tot_C), na.rm = TRUE)

  # make room on the right for the legend, restore settings afterwards
  old_par <- par(mar = c(5, 4, 4, 11))
  on.exit(par(old_par))

  plot(NA, xlim = c(0, x_max), ylim = c(0, y_max * 1.05),
       xlab = "Generation", ylab = "Population size",
       main = "Evolutionary rescue with habitat fragmentation")

  # carrying capacities: K only until the split, then K_B and K_C
  segments(0, K, split_time, K, col = "black", lty = 2, lwd = 2)
  segments(split_time, K_B, x_max, K_B, col = "darkgreen",  lty = 2, lwd = 2)
  segments(split_time, K_C, x_max, K_C, col = "darkorange", lty = 2, lwd = 2)

  # before the split: wild type and mutant
  lines(gen_main, wt_main,  col = "blue",      lwd = 2)
  lines(gen_main, mut_main, col = "firebrick", lwd = 2)

  # after the split: total of habitat B and C
  if (length(tot_B) > 0) lines(gen_B, tot_B, col = "darkgreen",  lwd = 2)
  if (length(tot_C) > 0) lines(gen_C, tot_C, col = "darkorange", lwd = 2)

  # split time
  abline(v = split_time, col = "grey40", lty = 3, lwd = 2)

  # legend outside, at the top right corner of the plot region
  legend(x = par("usr")[2], y = par("usr")[4], xpd = TRUE, bty = "n",
         xjust = 0, yjust = 1, cex = 0.85,
         legend = c("Wild type (before split)", "Mutant (before split)",
                    "Habitat B (total)", "Habitat C (total)",
                    "K (before split)", "K_B", "K_C", "Split time"),
         col = c("blue", "firebrick", "darkgreen", "darkorange",
                 "black", "darkgreen", "darkorange", "grey40"),
         lty = c(1, 1, 1, 1, 2, 2, 2, 3), lwd = 2)
}

# ---- 2. Single habitat panel (used for B and C) ----
plot_habitat <- function(df, K_hab, split_time, title, col_tot, xlim_max = NULL) {

  if (is.null(df) || nrow(df) == 0) {
    plot.new()
    title(main = paste(title, "- no data (population ended before the split)"))
    return(invisible(NULL))
  }
```

```r
  gens  <- split_time + seq_len(nrow(df))
  tot   <- df$a + df$A
  x_max <- if (is.null(xlim_max)) max(gens) else xlim_max
  y_max <- max(c(K_hab, tot), na.rm = TRUE)

  plot(NA, xlim = c(split_time, x_max), ylim = c(0, y_max * 1.05),
       xlab = "Generation", ylab = "Population size", main = title)

  abline(h = K_hab, col = col_tot, lty = 2, lwd = 2)       # K of this habitat
  abline(v = split_time, col = "grey40", lty = 3, lwd = 2) # split time

  lines(gens, tot,  col = col_tot,     lwd = 2)  # total
  lines(gens, df$a, col = "blue",      lwd = 2)  # wild type
  lines(gens, df$A, col = "firebrick", lwd = 2)  # mutant



  legend(x = par("usr")[2], y = par("usr")[4], xpd = TRUE, bty = "n",
         xjust = 0, yjust = 1, cex = 0.85,
         legend = c("Total", "Wild type (a)", "Mutant (A)", "K", "Split time"),
         col = c(col_tot, "blue", "firebrick", col_tot, "grey40"),
         lty = c(1, 1, 1, 2, 3), lwd = 2)

  ##adding the starting population
    legend(x = par("usr")[2], y = par("usr")[4], xpd = TRUE, bty = "n",
         xjust = 0, yjust = 1, cex = 0.85,
         legend = c("Total",
                    paste0("Wild type (a), start ", df$a[1]),
                    paste0("Mutant (A), start ", df$A[1]),
                    "K", "Split time"),
         col = c(col_tot, "blue", "firebrick", col_tot, "grey40"),
         lty = c(1, 1, 1, 2, 3), lwd = 2)
}

# ---- 3. Two panels: habitat B on top, habitat C below ----
plot_habitats <- function(output, K, habitat_prop, split_time, xlim_max = NULL) {

  K_B <- K * habitat_prop
  K_C <- K * (1 - habitat_prop)

  # two panels stacked, with room on the right for the legends
  old_par <- par(mfrow = c(2, 1), mar = c(4, 4, 3, 11))
  on.exit(par(old_par))

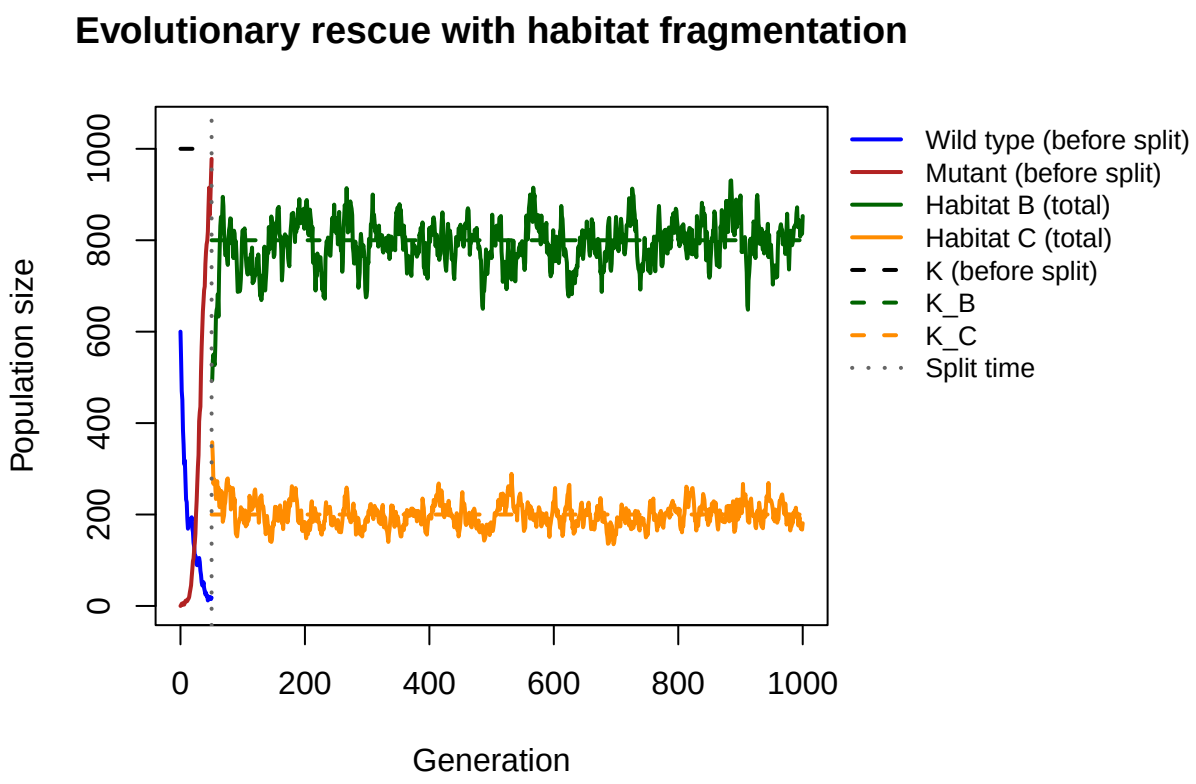
  plot_habitat(output$B, K_B, split_time, "Habitat B", "darkgreen",  xlim_max)
  plot_habitat(output$C, K_C, split_time, "Habitat C", "darkorange", xlim_max)
}

# ============================================================
# Usage
# (simulate_one_gen and simulate_pop must be defined first)
# ============================================================
max_gen      <- 1000
```

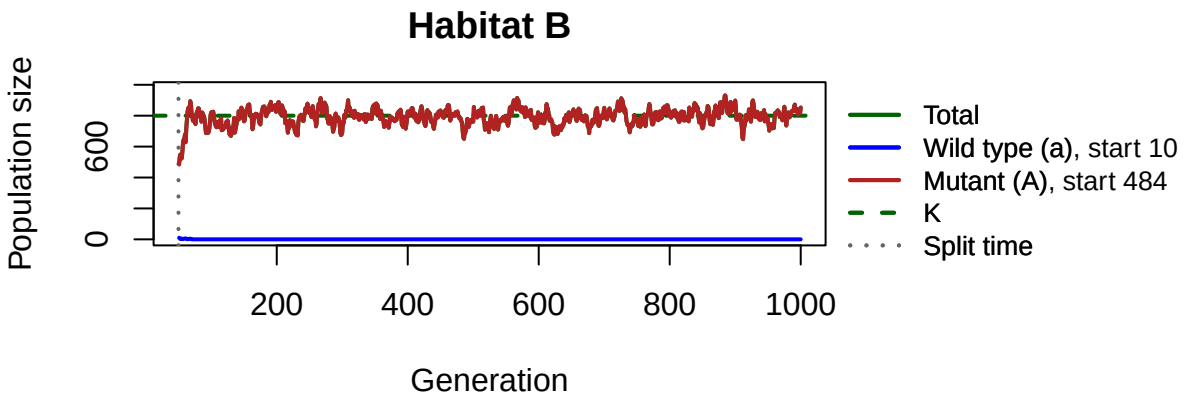
```r
K             <- 1000
habitat_prop <- 0.8
split_time    <- 50
split_ratio   <- 0.5

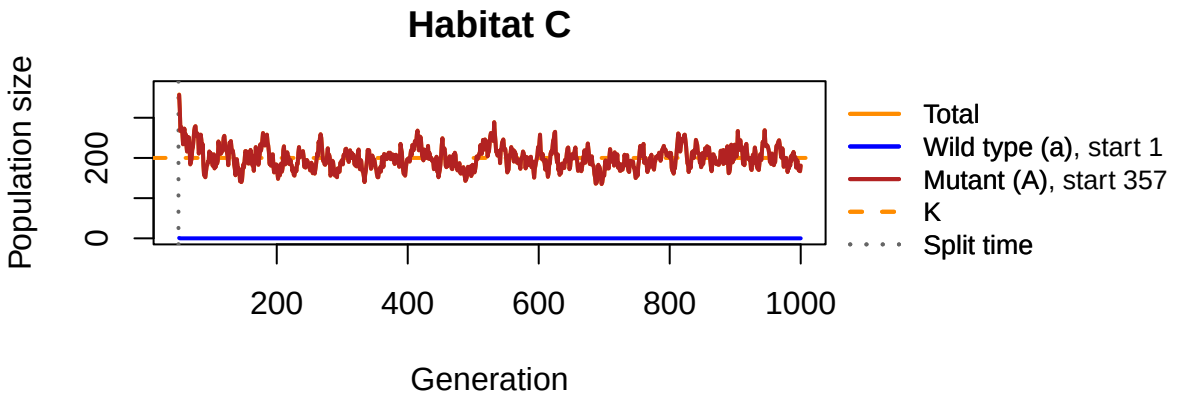
output <- simulate_pop(600, 0, 0.01, 0.2, 0.001, max_gen, K,
                       habitat_prop, split_ratio , split_time)

# overview plot
plot_split(output, K = K, habitat_prop = habitat_prop, split_time = split_time)
```

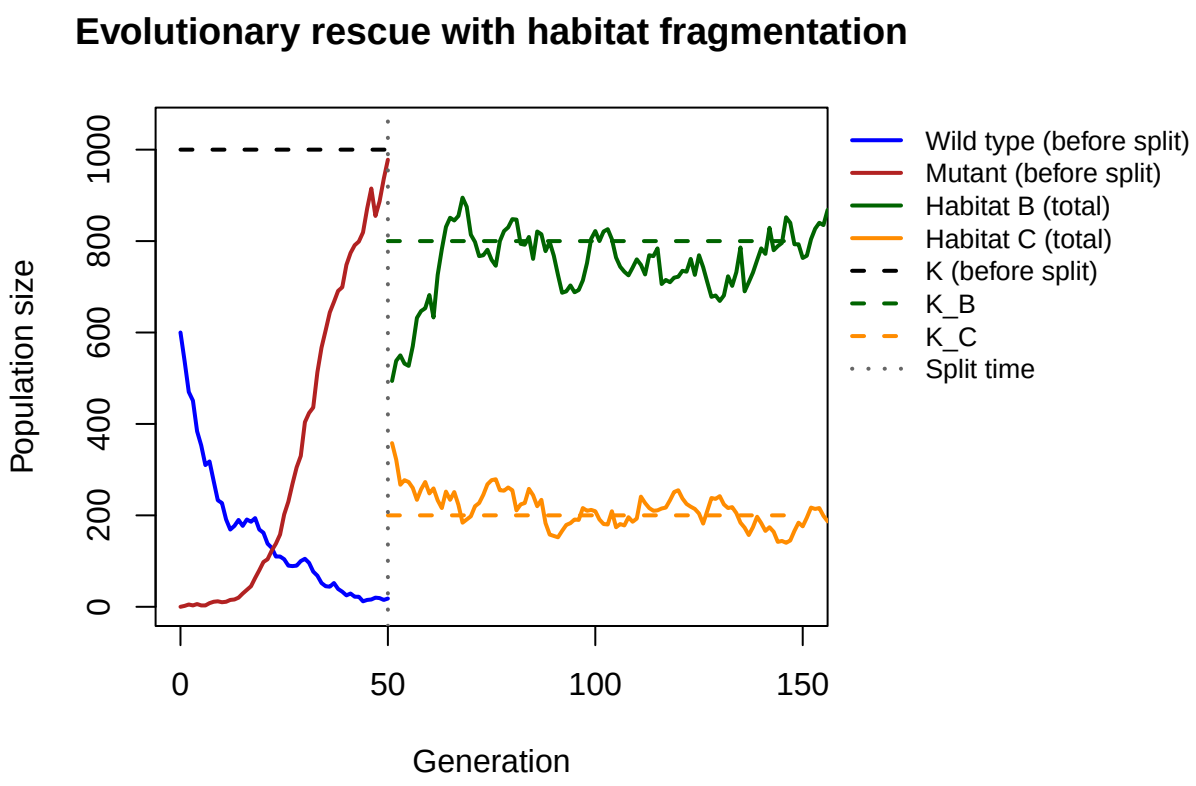



```r
# panels for habitat B and C
plot_habitats(output, K = K, habitat_prop = habitat_prop, split_time = split_time)
```

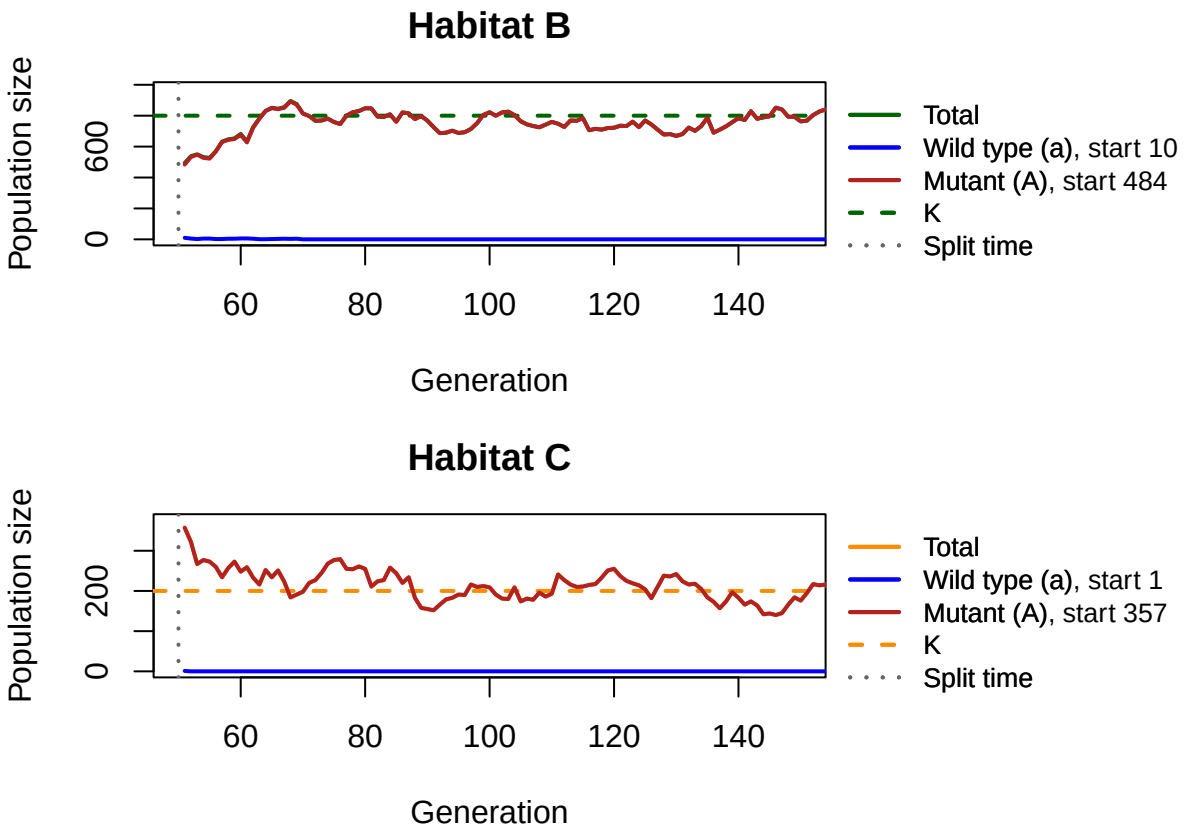





```
# zoomed in on the first 150 generations
plot_split(output, K = K, habitat_prop = habitat_prop,
           split_time = split_time, xlim_max = 150)
```




```
plot_habitats(output, K = K, habitat_prop = habitat_prop,
              split_time = split_time, xlim_max = 150)
```



Less common functions used

- `par()`: sets global graphics parameters. `mar` = margins (bottom, left, top, right) in lines; `mfrow = c(2, 1)` = a grid of 2 rows × 1 column of plots. `par()` returns the *old* values, which we save in `old_par`.
- `on.exit(par(old_par))`: registers code that runs automatically when the function ends, restoring the original graphics settings so that later plots are not affected.
- `plot(NA, ...)`: draws an empty coordinate system (nothing is plotted for `NA`); lines and segments are then added manually.
- `segments(x0, y0, x1, y1)`: draws a straight line between two points; used for the carrying-capacity lines that only span part of the x-axis.
- `abline(v = ..., h = ...)`: draws an infinite vertical (`v`) or horizontal (`h`) line.
- `lines(x, y)`: adds a line connecting the points to an existing plot.
- `legend()`: adds a legend. `par("usr")` returns the plot region's coordinates (x-min, x-max, y-min, y-max), so `par("usr")[2]` and `[4]` are the top-right corner. `xpd = TRUE` allows drawing outside the plot region, `bty = "n"` removes the legend box, `xjust`/`yjust` set which corner of the legend is anchored at that point.
- `seq_along(x)` / `seq_len(n)`: safe ways to create `1, 2, ..., length(x)` or `1, ..., n` (they return an empty sequence for empty input, unlike `1:length(x)`).
- `if (...) a else b` used as an expression: `x_max <- if (is.null(xlim_max)) ... else ...` assigns one of two values.
- `plot.new()` + `title()`: create a blank plot and add a title, used as a placeholder when there is no data.
- `invisible(NULL)`: returns `NULL` without printing it.
- `paste0()`: joins strings without separator, used to put the starting numbers in the legend.

A small analysis

As you can see if you simulate and plot the output several times, there is a lot of stochasticity/variation in the outcome. Therefore, to get interpretable results, we have to run the simulation many times and summarize the outcome depending on the question of our study. Here I am showing an example in which we record the time and the value of the lowest population size (or the time of extinction, if the population size eventually goes to 0). Usually, we want to study this for a range of model parameters; here I am varying the decay rate and the selection coefficient. Other (maybe more interesting) features to study would be the following:

- rescue probability dependent on initial frequency and selection coefficient
- time at which the rescue mutation occurs first if it rescues, vs. if it doesn't rescue, dependent on initial population size and mutation rate
- time until the population has recovered in case it becomes rescued, dependent on mutation rate and selection coefficient
- ...

```
library(ggplot2)
```

```
## Warning: Paket 'ggplot2' wurde unter R Version 4.5.3 erstellt
```

```
library(tidyr)
library(dplyr)
```

```
## Warning: Paket 'dplyr' wurde unter R Version 4.5.3 erstellt
```

```
##
## Attache Paket: 'dplyr'
```

```
## Die folgenden Objekte sind maskiert von 'package:stats':
##
##     filter, lag
```

```
## Die folgenden Objekte sind maskiert von 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
# ---- fixed parameters ----------------------------------------------------
init_a           <- 600
init_A           <- 0
m_rate           <- 0.001
max_gen_analysis <- 1000
K_analysis       <- 1000
no_replicates    <- 1    # more replicates give more precise probabilities
max_attempts     <- 5000
rescue_frac      <- 1       # rescue = A >= rescue_frac * K of the habitat, changeable

# ---- core function: your loop, wrapped so it can be reused -------------
run_scenarios <- function(split_times, habitat_props) {
  results <- list()

  for (st in split_times) {
```

```
  for (hp in habitat_props) {

    K_B <- K_analysis * hp
    K_C <- K_analysis * (1 - hp)

    i <- 0
    attempts <- 0
    repeat {
      attempts <- attempts + 1
      if (attempts > max_attempts) {
        warning(paste("Too many failed runs for st =", st, "hp =", hp))
        break
      }

      one_run <- simulate_pop(
        init_a, init_A,
        decay_rate   = 0.01,
        sel_coeff    = 0.2,
        mut_rate     = m_rate,
        t_max        = max_gen_analysis,
        K            = K_analysis,
        habitat_prop = hp,
        split_ratio  = 0.5,
        split_time   = st
      )

      # skip runs that ended before the split time
      if (nrow(one_run$B) == 0 || nrow(one_run$C) == 0) next
      i <- i + 1

      reached_B <- one_run$B$A >= rescue_frac * K_B
      reached_C <- one_run$C$A >= rescue_frac * K_C
      rescued_B <- any(reached_B)
      rescued_C <- any(reached_C)

      results[[length(results) + 1]] <- data.frame(
        st = st,
        hp = hp,
        replicate = i,
        rescued_B = rescued_B,
        rescued_C = rescued_C,
        rescued_any  = rescued_B | rescued_C,
        rescued_both = rescued_B & rescued_C,
        gen_B = if (rescued_B) which(reached_B)[1] else NA_integer_,
        gen_C = if (rescued_C) which(reached_C)[1] else NA_integer_
      )

      if (i >= no_replicates) break
    }
  }
}

data_table <- do.call(rbind, results)
```

```r
  # rescue probability per scenario, with binomial standard error
  rescue_prob <- data_table %>%
    group_by(st, hp) %>%
    summarise(
      n = n(),
      across(c(rescued_B, rescued_C, rescued_any, rescued_both), mean),
      .groups = "drop"
    )

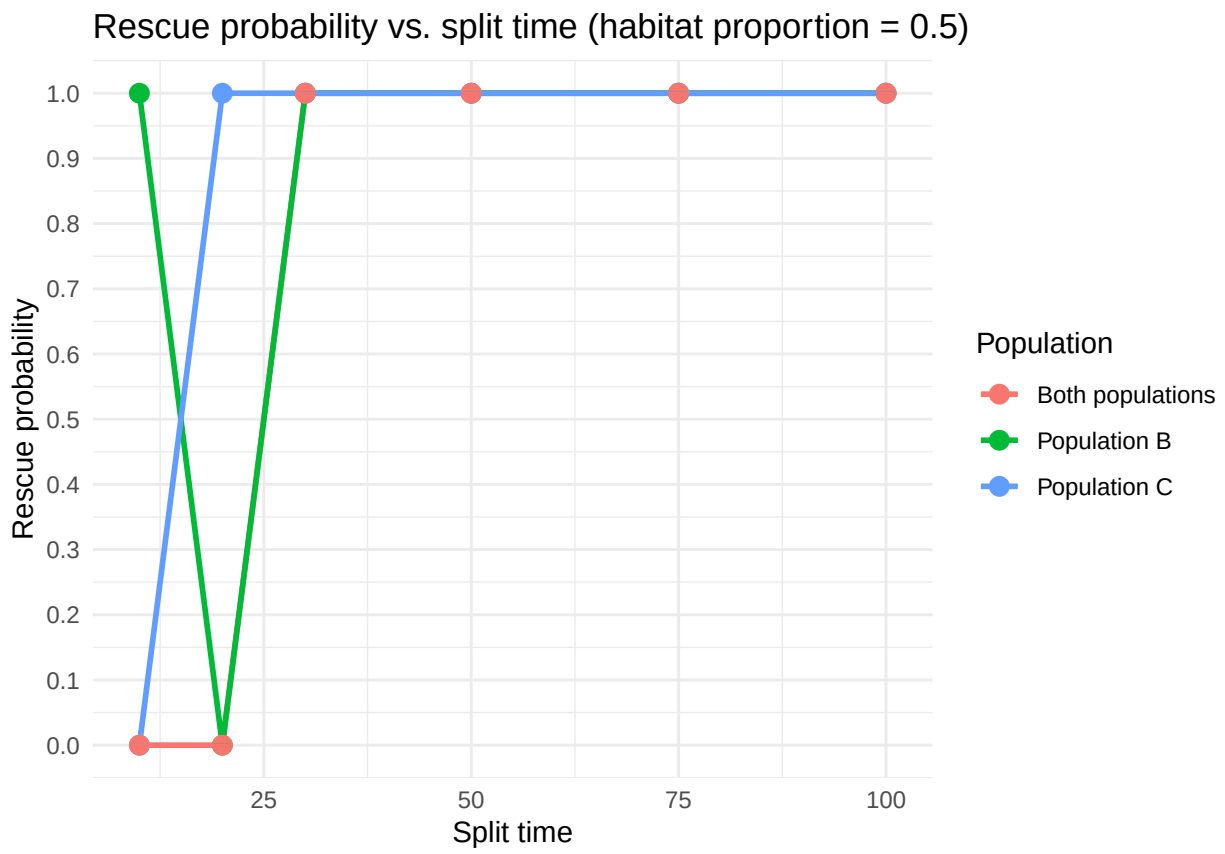
  list(data = data_table, prob = rescue_prob)
}

# ---- helper: long format with SE for plotting --------------------------
to_long <- function(rescue_prob) {
  rescue_prob %>%
    pivot_longer(
      cols = c(rescued_B, rescued_C, rescued_both),
      names_to = "population",
      values_to = "rescue_probability"
    ) %>%
    mutate(
      se = sqrt(rescue_probability * (1 - rescue_probability) / n),
      population = recode(
        population,
        rescued_B    = "Population B",
        rescued_C    = "Population C",
        rescued_both = "Both populations"
      )
    )
}

# ======================================================================
# ANALYSIS 1: split time varies, habitat proportion fixed
# ======================================================================
res1 <- run_scenarios(
  split_times   = c(10, 20, 30, 50, 75, 100),
  habitat_props = 0.5
)
long1 <- to_long(res1$prob)
print(res1$prob)
```

```
## # A tibble: 6 x 7
##      st    hp     n rescued_B rescued_C rescued_any rescued_both
##   <dbl> <dbl> <int>     <dbl>     <dbl>       <dbl>        <dbl>
## 1    10   0.5     1         1         0           1            0
## 2    20   0.5     1         0         1           1            0
## 3    30   0.5     1         1         1           1            1
## 4    50   0.5     1         1         1           1            1
## 5    75   0.5     1         1         1           1            1
## 6   100   0.5     1         1         1           1            1
```

```r
ggplot(long1, aes(x = st, y = rescue_probability,
                  color = population, group = population)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = pmax(0, rescue_probability - se),
                    ymax = pmin(1, rescue_probability + se)),
                width = 2) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.1)) +
  labs(title = "Rescue probability vs. split time (habitat proportion = 0.5)",
       x = "Split time", y = "Rescue probability", color = "Population") +
  theme_minimal()
```

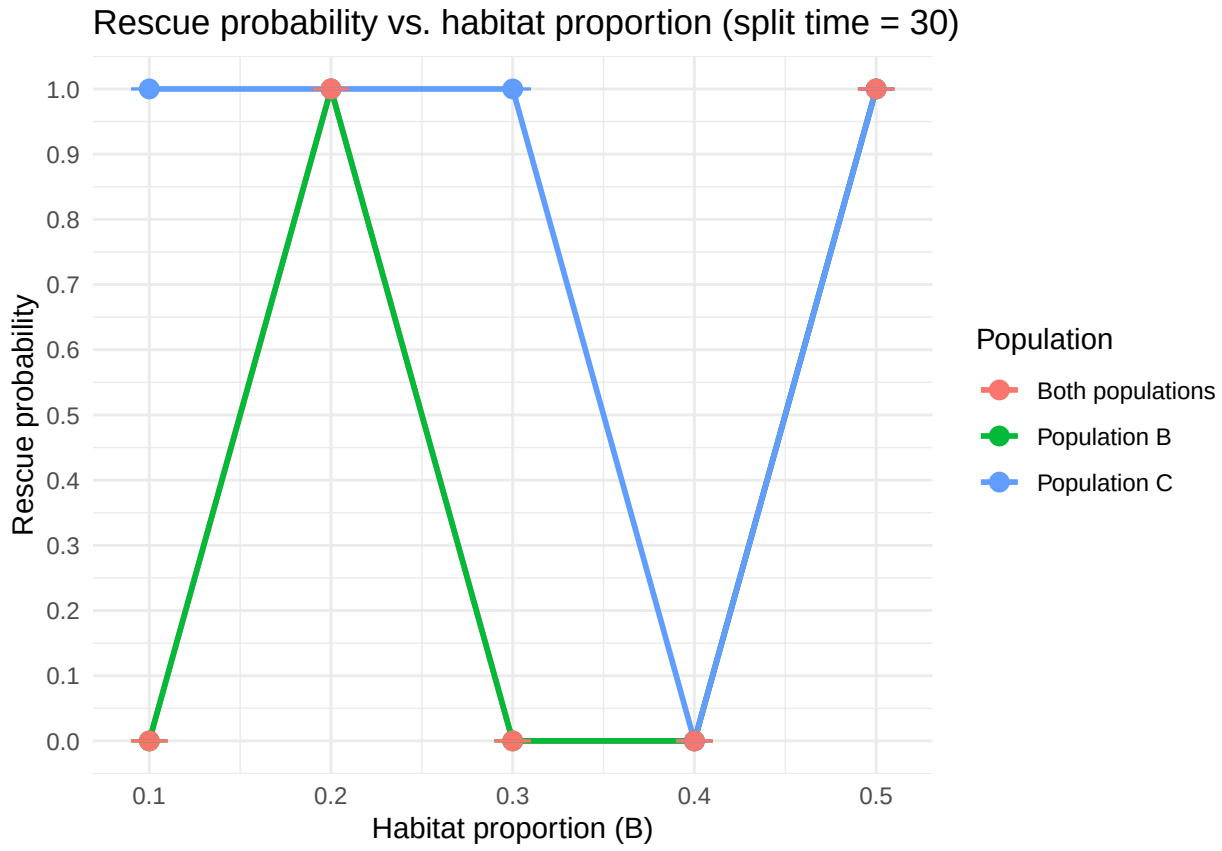



```r
# ========================================================================
# ANALYSIS 2: habitat proportion varies, split time fixed
# ========================================================================
res2 <- run_scenarios(
  split_times   = 30,
  habitat_props = c(0.1, 0.2, 0.3, 0.4, 0.5)
)
long2 <- to_long(res2$prob)
print(res2$prob)
```

```
## # A tibble: 5 x 7
##      st    hp     n rescued_B rescued_C rescued_any rescued_both
##   <dbl> <dbl> <int>     <dbl>     <dbl>       <dbl>        <dbl>
```

```
## 1    30   0.1     1         0         1           1           0
## 2    30   0.2     1         1         1           1           1
## 3    30   0.3     1         0         1           1           0
## 4    30   0.4     1         0         0           0           0
## 5    30   0.5     1         1         1           1           1
```

```r
ggplot(long2, aes(x = hp, y = rescue_probability,
                  color = population, group = population)) +
  geom_line(linewidth = 1) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = pmax(0, rescue_probability - se),
                    ymax = pmin(1, rescue_probability + se)),
                width = 0.02) +
  scale_x_continuous(breaks = seq(0.1, 0.9, 0.1)) +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.1)) +
  labs(title = "Rescue probability vs. habitat proportion (split time = 30)",
       x = "Habitat proportion (B)", y = "Rescue probability",
       color = "Population") +
  theme_minimal()
```



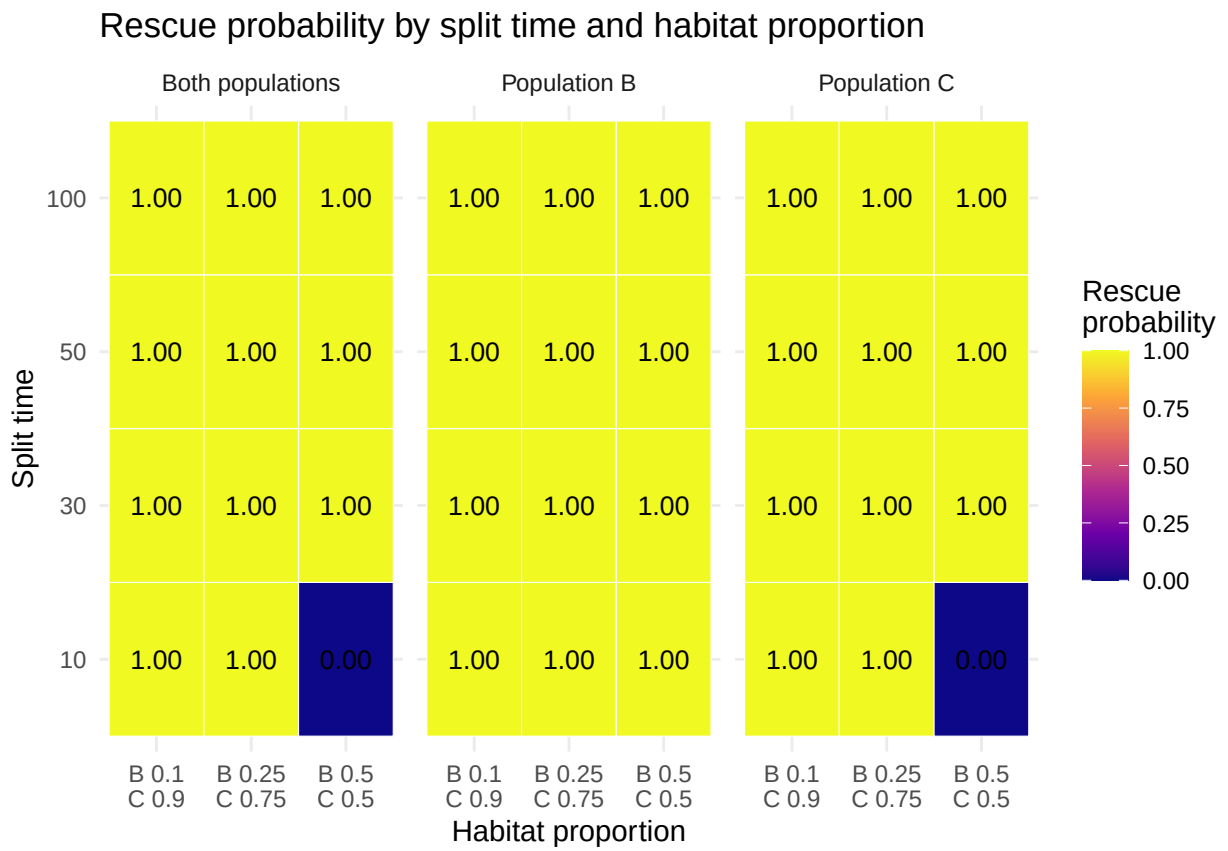

```r
# ============================================================================
# ANALYSIS 3: both vary (full grid)
# ============================================================================
res3 <- run_scenarios(
  split_times   = c(10, 30, 50, 100),
```

```
  habitat_props = c(0.1, 0.25, 0.5)
)
long3 <- to_long(res3$prob)
print(res3$prob)
```

```
## # A tibble: 12 x 7
##       st    hp     n rescued_B rescued_C rescued_any rescued_both
##    <dbl> <dbl> <int>     <dbl>     <dbl>       <dbl>        <dbl>
##  1    10  0.1      1         1         1           1            1
##  2    10  0.25     1         1         1           1            1
##  3    10  0.5      1         1         0           1            0
##  4    30  0.1      1         1         1           1            1
##  5    30  0.25     1         1         1           1            1
##  6    30  0.5      1         1         1           1            1
##  7    50  0.1      1         1         1           1            1
##  8    50  0.25     1         1         1           1            1
##  9    50  0.5      1         1         1           1            1
## 10   100  0.1      1         1         1           1            1
## 11   100  0.25     1         1         1           1            1
## 12   100  0.5      1         1         1           1            1
```

```
# heatmap: one panel per population
# label showing the proportion of BOTH habitats on two lines, e.g. "B 0.1" over "C 0.9"
hp_values <- sort(unique(long3$hp))
long3 <- long3 %>%
  mutate(hp_label = factor(hp, levels = hp_values,
                           labels = paste0("B ", hp_values, "\nC ", 1 - hp_values)))

ggplot(long3, aes(x = hp_label, y = factor(st), fill = rescue_probability)) +
  geom_tile(color = "white") +
  geom_text(aes(label = sprintf("%.2f", rescue_probability)), size = 3.5) +
  facet_wrap(~ population) +
  scale_fill_viridis_c(limits = c(0, 1), option = "C") +
  labs(title = "Rescue probability by split time and habitat proportion",
       x = "Habitat proportion", y = "Split time",
       fill = "Rescue\nprobability") +
  theme_minimal()
```

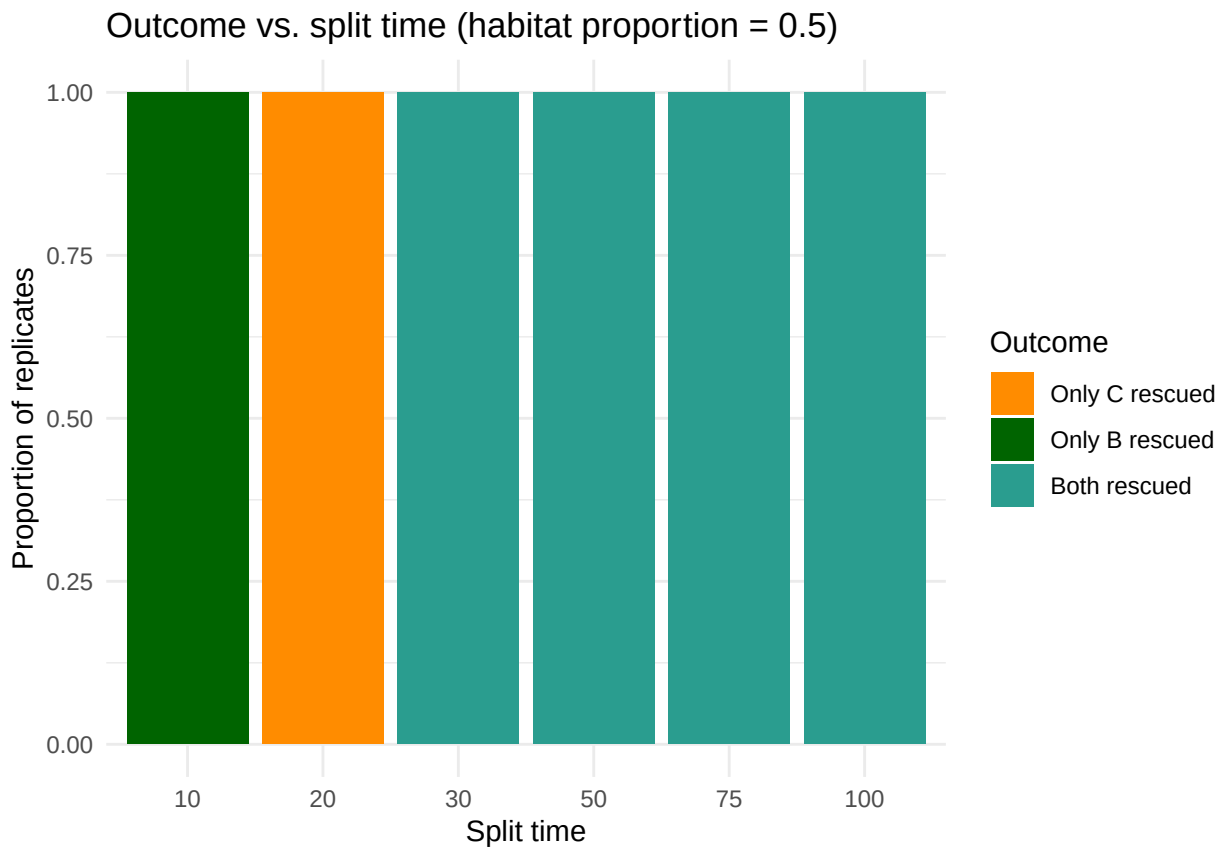



```r
# ======================================================================
# ANALYSIS 4: both vary (bar plots percentage)
# ======================================================================
outcome_cols <- c("Both rescued"    = "#2a9d8f",
                  "Only B rescued"  = "darkgreen",
                  "Only C rescued"  = "darkorange",
                  "Neither rescued" = "grey75")

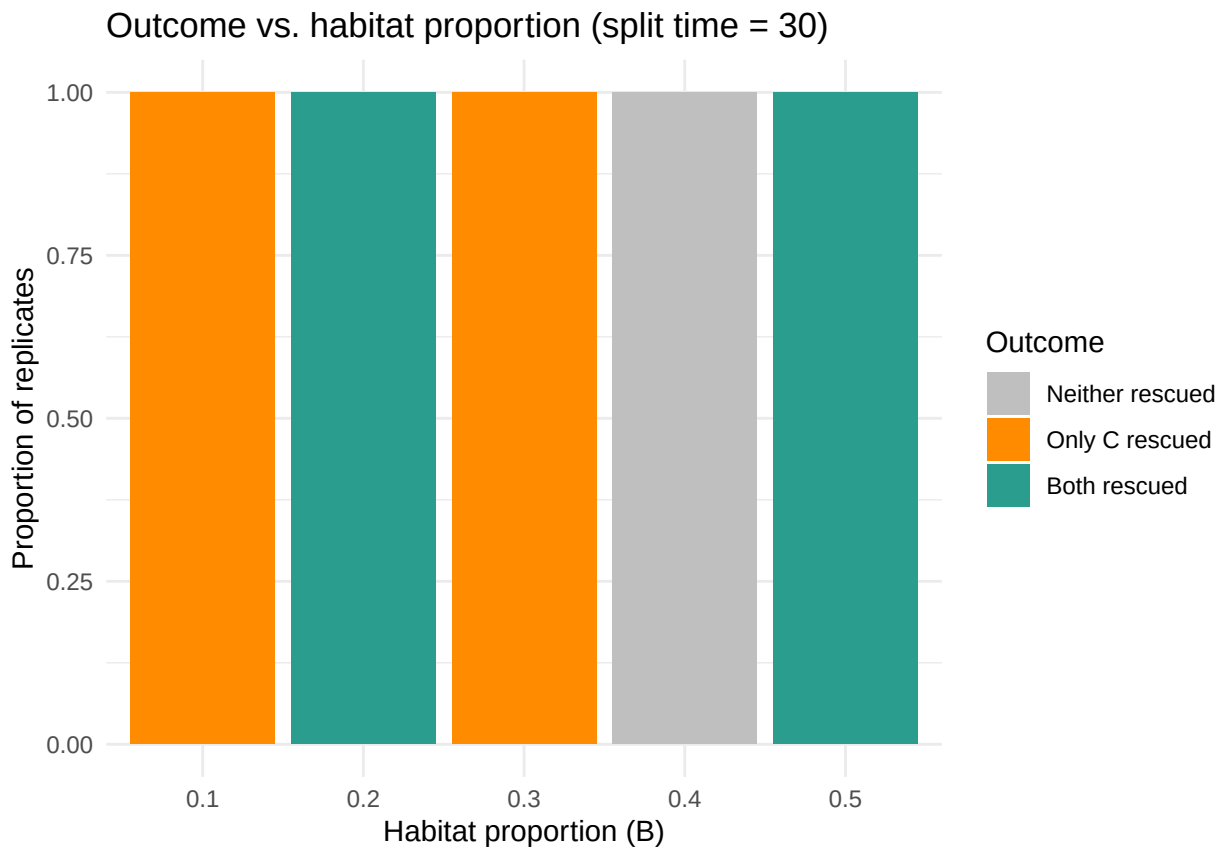
plot_outcomes <- function(dat, x_var, x_label, title) {
  dat <- dat %>%
    mutate(outcome = case_when(
      rescued_both ~ "Both rescued",
      rescued_B    ~ "Only B rescued",
      rescued_C    ~ "Only C rescued",
      TRUE         ~ "Neither rescued"),
      outcome = factor(outcome, levels = names(outcome_cols)[c(4, 3, 2, 1)]))

  ggplot(dat, aes(x = factor(.data[[x_var]]), fill = outcome)) +
    geom_bar(position = "fill") +          # each bar sums to 1 (= 100 %)
    scale_fill_manual(values = outcome_cols) +
    labs(title = title, x = x_label, y = "Proportion of replicates", fill = "Outcome") +
    theme_minimal()
}

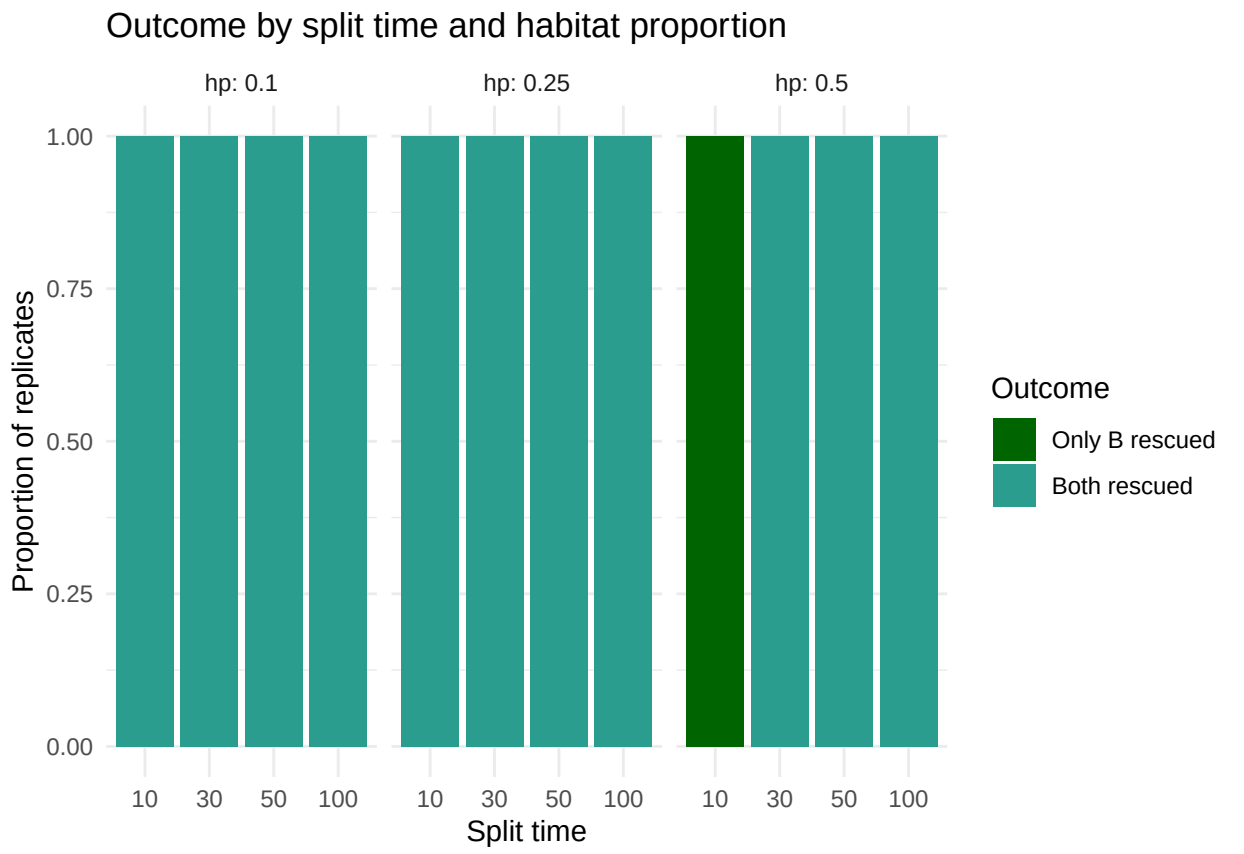
plot_outcomes(res1$data, "st", "Split time", "Outcome vs. split time (habitat proportion = 0.5)")
```




```
plot_outcomes(res2$data, "hp", "Habitat proportion (B)", "Outcome vs. habitat proportion (split time = 3
```




```
plot_outcomes(res3$data, "st", "Split time", "Outcome by split time and habitat proportion") +
  facet_wrap(~ hp, labeller = label_both)
```




```
#bettter legends
```

Appendix

Less common functions used

- `set.seed(57)`: initialises R's random number generator. With the same seed, all "random" draws are repeated identically, so results can be reproduced. Without it every knit gives different numbers.
- `rpois(n, lambda)`: draws `n` random numbers from a Poisson distribution with mean `lambda`. The Poisson distribution describes counts of independent events, which is a natural model for the number of offspring.
- `max(0, x)`: returns the larger of the two values, used here as a floor at 0.
- `rbinom(n, size, prob)`: draws `n` random numbers from a binomial distribution, i.e. the number of "successes" among `size` independent trials, each with success probability `prob`. Here: how many of the individuals are assigned to habitat B.
- `rbind(x, y)`: "row-bind", appends `y` as a new row below `x`. Starting from `NULL`, the first `rbind` creates the matrix. This is how the time series grows by one row per generation.
- `rownames()` / `colnames()`: set the labels of the rows / columns of a matrix. `(0:t_max)[1:length(...)]` creates the generation numbers and cuts them to the actual number of rows.
- `break`: leaves the `for` loop immediately (used for early stopping).
- `|`: element-wise logical "or" (here equivalent to `||` since both sides are single values).
- `as.data.frame()`: converts a matrix into a data frame, so columns can be accessed with `$` (e.g. `output$B$A`). `as.data.frame(NULL)` gives an empty data frame with 0 rows, which is how we detect runs that ended before the split.

- `list(main = ..., B = ..., C = ...)`: returns several objects of different size in one named container.
- `par()`: sets global graphics parameters. `mar` = margins (bottom, left, top, right) in lines; `mfrow = c(2, 1)` = a grid of 2 rows × 1 column of plots. `par()` returns the *old* values, which we save in `old_par`.
- `on.exit(par(old_par))`: registers code that runs automatically when the function ends, restoring the original graphics settings so that later plots are not affected.
- `segments(x0, y0, x1, y1)`: draws a straight line between two points; used for the carrying-capacity lines that only span part of the x-axis.
- `legend()`: adds a legend. `par("usr")` returns the plot region's coordinates (x-min, x-max, y-min, y-max), so `par("usr")[2]` and `[4]` are the top-right corner. `xpd = TRUE` allows drawing outside the plot region, `bty = "n"` removes the legend box, `xjust`/`yjust` set which corner of the legend is anchored at that point.
- `seq_along(x)` / `seq_len(n)`: safe ways to create `1, 2, ..., length(x)` or `1, ..., n` (they return an empty sequence for empty input, unlike `1:length(x)`).
- `if (...) a else b` used as an expression: `x_max <- if (is.null(xlim_max)) ... else ...` assigns one of two values.
- `plot.new()` + `title()`: create a blank plot and add a title, used as a placeholder when there is no data.
- `invisible(NULL)`: returns `NULL` without printing it.
- `paste0()`: joins strings without separator, used to put the starting numbers in the legend.
- `repeat { ... }`: an endless loop that is left only with `break`. Used here because we do not know in advance how many attempts are needed to get enough valid runs.
- `next`: skips the rest of the current loop iteration and goes to the next one.
- `any(x)`: `TRUE` if at least one element of the logical vector `x` is `TRUE`.
- `which(x)[1]`: `which()` returns the positions of all `TRUE` elements; `[1]` takes the first, i.e. the **first generation** at which the threshold was reached. Note this counts rows after the split (row 1 = first generation after the split).
- `NA_integer_`: a missing value of integer type (a plain `NA` could create type conflicts when binding rows).
- `results[[length(results) + 1]] <- ...`: appends an element to the end of a list.
- `do.call(rbind, results)`: calls `rbind` with the list elements as separate arguments, i.e. stacks all data frames in the list into one big table.
- `%>%` (pipe): passes the result on the left as the first argument of the function on the right, so operations can be read top to bottom.
- `group_by(st, hp)`: tells `dplyr` to do the following calculations separately for each combination of `st` and `hp`.
- `summarise(...)`: collapses each group into a single row. `n()` counts the rows in the group; `across(cols, mean)` applies `mean` to several columns at once; `.groups = "drop"` removes the grouping afterwards.
- `pivot_longer()`: turns several columns into two (`names_to` gets the old column names, `values_to` the values). One row per scenario and population results, as `ggplot2` needs..
- `ggplot(data, aes(...)) + geom_*()`: `ggplot2` builds a plot in layers. `aes()` maps data columns to visual properties (x, y, colour, fill), and each `geom_` adds a layer: `geom_line`, `geom_point`, `geom_errorbar` (error bars), `geom_tile` (coloured rectangles), `geom_text` (printed numbers).
- `pmax()` / `pmin()`: *parallel* maximum / minimum, applied element by element to a vector (unlike `max()`, which returns a single number). Used to keep error bars inside [0, 1].
- `scale_y_continuous(limits, breaks)`: sets the axis range and tick positions. `seq(from, to, by)` generates the tick values.
- `labs()`: sets titles and axis/legend labels. `theme_minimal()`: applies a clean plot theme.
- `facet_wrap(~ population)`: splits the plot into one panel per value of `population`.
- `scale_fill_viridis_c(option = "C")`: a continuous colour scale (viridis palette), readable also for colour-blind people.
- `factor(x, levels, labels)`: converts values into a categorical variable with a fixed order and custom

display labels. `factor(st)` makes the split time a category so that tiles are evenly spaced.

- `sprintf("%.2f", x)`: formats a number as text with 2 decimals.
- `sort(unique(x))`: the distinct values of `x` in ascending order.